

JOBSHEET 9

STACK

Nama:Daffa Dwi Nugroho

Kelas:TI-1C/10

2.1 Percobaan 1: Mahasiswa Mengumpulkan Tugas

2.1.3 Pertanyaan

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

Yang perlu diperbaiki dibagian ini

```
public void print(){
    for (int i = 0; i <= top; i++) {
        System.out.println(stack[i].nama + "\t" + stack[i].nim+ "\t" + stack[i].kelas);
    }
}
```

Menjadi ini dikarenakan logika di kode awal salah, dengan menerapkan last in, first out, perurutannya dibalik jika menggunakan kode awal itu salah dari perurutannya menjadi seperti ini

```
public void print(){
    for (int i = top; i >= 0; i--) {
        System.out.println(stack[i].nama + "\t" + stack[i].nim+ "\t" + stack[i].kelas);
    }
}
```

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!

Banyak data yang dapat ditampung ialah 5 data

```
public static void main(String[] args) {
    Stacktugasmahasiswa10 stack = new Stacktugasmahasiswa10(size: 5);
}
```

3.Mengapa perlu pengecekan kondisi !isFull() pada method push? Kalau kondisi if-else tersebut dihapus, apa dampaknya?

Diperlukan kondisi tersebut dikarenakan program harus mengecek terlebih dahulu apakah sudah full atau belum jika full maka tidak bisa dijalankan program tersebut dan sebaliknya jg, jika kondisi tersebut dihapus akan menyebabkan tidak diketahui apakah program full atau tidak

4. Modifikasi kode program pada class MahasiswaDemo dan StackTugasMahasiswa sehingga pengguna juga dapat melihat mahasiswa yang pertama kali mengumpulkan tugas melalui operasi lihat tugas terbawah!

```
        break;
    case 5:
        Mahasiswa10 bawah = stack.peekBottom();
        if (bawah != null) {
            System.out.println("Tugas pertama dikumpulkan oleh " + bawah.nama);
        }
        break;
```

```
    public Mahasiswa10 peekBottom() {
        if (!isEmpty()) {
            return stack[0];
        } else {
            System.out.println(x: "Stack kosong! Tidak ada tugas yang dikumpulkan");
            return null;
        }
    }
}
```

5. Tambahkan method untuk dapat menghitung berapa banyak tugas yang sudah dikumpulkan saat ini, serta tambahkan operasi menunya!

Karena yang diminta hanya berapa banyak tugas jadi kita bisa hanya menambahkan ini

```
    case 6:
        int jumlah = stack.JumlahTugas();
        System.out.println("Jumlah tugas yang telah dikumpulkan saat ini: " + jumlah);
        break;

    public int JumlahTugas() {
        return top + 1;
    }
}
```

2.2 Percobaan 2: Konversi Nilai Tugas ke Biner

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!

alur kerja tersebut memanfaatkan prinsip stack yaitu last in first out, hal ini sama seperti saat kita menghitung biner manual dimana sisa bagi paling akhir ditulis di awal, nah di program ini kita bisa memanfaatkan dengan saat melakukan kondisi nilai >0 program akan mencari sisa hasil bagi yang sisal bagi tersebut mnjdi nilai terakhir sendiri , hal tersebut diulang hingga selesai.

2. Pada method konversiDesimalKeBiner, ubah kondisi perulangan menjadi while (kode $!= 0$), bagaimana hasilnya? Jelaskan alasannya!

Karena nilai tugas mahasiswa yang diinputkan selalu berupa bilangan positif maka kondisi > 0 dan $\neq 0$ akan memberikan efek berhenti yang sama.

Pembagian nilai akan terus dilakukan hingga angkanya habis mencapai 0.

Keduanya sama-sama akan menghentikan perulangan tepat saat nilai tersebut menjadi 0, sehingga jika kodenya diubah tidak akan terjadi perubahan apa-apa pada hasil akhirnya."